K215: Shorter Proofs Through Shared Invariants and Exact Runtime Contracts

A source-based research and implementation note for ProveIt

21 October 2026

Reviewed revision: `57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`

Abstract

The existing K215 endpoint already has a short machine-specific facade. The productive next reductions are in the reusable middle layers: share the implicational derivation, strengthen a dense/sparse simulation invariant so that one induction proves both directions, and replace repeated decoder execution with an exact prefix theorem followed by occurrence-specific residual checks. This packet develops these recommendations against the actual merged source tree. It distinguishes current production code, previous proposals, newly checked component proofs, and remaining endpoint integration. It also supplies a constructive finite-cleanup simulation theorem in Lean and Coq, explaining both its benefit and its present source-size overhead. The literal 215-state machine and its consistency-equivalence statement are preserved.

1 Recommendation and contribution

Adopt the small one-invariant dense bridge and the existing shared implication derivation first. Integrate the certified-prefix/residual-check component as the next runtime improvement. Transfer reusable interfaces between Lean and Rocq only where they remove an actual duplication.

Three distinct measures matter here. A smaller hand-maintained argument is easier to review. A smaller generated proof source can reduce elaboration and maintenance cost. A smaller finite computation may reduce build work. These measures must be recorded separately: a newly shared theorem can simplify reasoning while increasing the total number of source lines during the first adoption.

The immediate Lean proposal has two independent parts. The first is a new local replacement for `KDenseBridge.blank_iff`: retain both machines' halting facts in the stopped branch of the induction invariant. Its original second induction becomes unnecessary. The second is a recheck and packaging of the repository's previously proposed six-goal implication core and unrestricted adapter, taken from commit `49e99c5b7cc6c2e164331961de28bad3904361fd` [5]. The latter is explicitly credited as existing work; a merge retained the earlier checked files instead of adopting it.

The same three files fall from 50,581 to 25,566 bytes, a reduction of 25,015 bytes (49.5%). Their physical-line reduction is 43.1%. The full selected endpoint source closure is much larger: 2,129,439 bytes and 41,170 lines. Consequently this immediate three-file change represents about 1.17% of that baseline closure's bytes, before any runtime-interface additions. Large percentages for the selected files must not be described as the percentage reduction of the complete formalization.

Selected Lean proof module	Current lines	Proposed lines	Change
`DominatedImplication`	287	300	+13
`Lukasiewicz`	524	101	−423
`KDenseBridge`	193	170	−23
Total for these three files	1,004	571	−433

Table 1: Physical source lines, including comments and blank lines. The shared core grows slightly while eliminating the separate implication derivation. Generator and test files are outside this three-file comparison.

The runtime proposal addresses a different cost. A finite source has 4,751 leaf occurrences and 6,176 guarded outcomes because decrement leaves have two outcomes. The maintained whole-site checker repeats decoder work for these occurrences. An already certified exact decoder prefix can be shared, while a residual checker retains every occurrence's complete path, branch condition, register transformation, and return target. This packet closes the computable-port selection interface and completes the kernel-checked residual coverage for all 6,176 outcomes. The resulting semantic source/table equivalence remains a component of the full endpoint.

Validation ledger. The following scopes are separate; no row denotes a fresh complete ZFC endpoint build.

Component	Completed check
Minimal Lean dense bridge	Fresh four-module component, matched original/candidate typed audit, identical public theorem signature and axiom output.
Shared implication basis	Fresh 16-module component including current numerical and Tarski clients. All six core exports and four unrestricted exports are axiom-free. Exact producer regeneration and static API comparison also pass.
Optional Lean cleanup interface	Fresh 19-module component including the actual dense bridge, register simulator, adapters, and three boundary examples.
Coq cleanup and relabeling	Nine source modules compile; recursive `coqchk -silent -o` succeeds for both endpoints. The named cleanup and concrete relabeling theorems are closed under the global context.
Decoder residual interface	All 4,751 leaves and 6,176 guarded outcomes are kernel-checked, with closed semantic coverage and a source/table halting-equivalence facade. The 51-module component completed across recorded resumed invocations.
Subtree-path abbreviation	All 105 statement rewrites and the ten-file source measurement were independently replayed statically; patch applicability checked. The Lean changes remain uncompiled.
Existing Rocq implication alternatives	Exact source comparison and patch applicability checked; their earlier repository receipts are attributed. These two replacements were not freshly compiled in this packet.

Lean checks use 4.32.0; Coq component checks use 8.18.0. Receipts bind sources, compiler results, and raw logs. Full machine-to-ZFC integration and the production Rocq 9.1.0 gate remain after adoption.

2 What the current repository actually proves

2.1 The literal endpoint

The maintained Lean statement is

$$\neg\,\texttt{BlankHalts machine215} \quad\longleftrightarrow\quad \neg\,\texttt{BProv ZFCax_s [] Form.fBot}.$$

The positive version equates blank-tape halting with a finite ZFC refutation. The negated equivalence is obtained by `not_congr`. The machine-specific facade composes five inputs: dense/sparse table agreement, register-routine contracts, the boot run, source-site coverage, and source-program calibration. The generic endpoint then passes through derivability of code 1 in `C222`, syntactic ZF refutability, and the ZF/ZFC refutation equivalence [2, 3].

This is the exact interface the proposed refactors preserve. The existing proof-assistant endpoint is an external theorem of the assistant. The repository separately records that its internally encoded PA endpoint remains open [4]. This work does not supply an internally encoded PA or ZFC derivation, and it does not prove either side of the displayed equivalence unconditionally. No change to a proof presentation should silently enlarge that claim.

2.2 The source snapshot and the merge issue

A static import traversal at the pinned revision finds 179 project modules, including `KEndpoint215`, with 2,129,439 source bytes and 41,170 physical lines. The only external directly named module is `Init`. The traversal uses source roots from `lakefile.toml`; its per-module paths, imports, lengths, and SHA-256 values are included in the evidence. This is a source-graph measurement, not a measurement of the kernel's minimal theorem dependencies.

Recent branch histories contain reports of 168- and 169-module closures and a 300-line shared implication core plus a 101-line adapter. Those numbers do not describe the actual pinned merge tree. The current production files are the 287-line core and 524-line `Lukasiewicz`; the 179-module traversal agrees with the phase-twelve checkpoint [4]. It is therefore appropriate to offer the six-goal core as a pending adoption, rather than announce it as a newly discovered or already merged reduction.

The same distinction applies to Rocq. Its endpoint has a very short final rewrite proof, while the retained checked lane still contains the older implication certificates and offset-oriented source facade. Earlier local proof receipts for alternative sources do not automatically certify a different current source hash.

2.3 A useful accounting discipline

For every proposed reduction, record four scopes:

1. the theorem statement and assumptions that remain fixed;
2. the source files and generator inputs that change;
3. the exact freshly compiled component and its unchanged callers;
4. any whole-endpoint rebuild or production-toolchain check that remains to be done.

The archive follows this discipline. It separates checked local replacements from new reusable infrastructure and from existing proposals that are only recommended for later production adoption.

3 Share the six-goal implication derivation

3.1 The mathematical interface

The dominated core deliberately uses little structure. Let A be a type with a binary operation $x \Rightarrow y$ and a relation $\preceq$ satisfying

$$x \preceq x, \qquad u \preceq x \Longrightarrow u \preceq (x \Rightarrow y), \qquad u \preceq y \Longrightarrow u \preceq (x \Rightarrow y).$$

Let $T \subseteq A$ be closed under modus ponens and contain the restricted rows

$$\rho(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)) \quad \text{whenever } u \preceq a.$$

The checked condensed-detachment derivation supplies the unrestricted row, composition, right-prefix lifting, K, Peirce, and S. It needs no decoding of A as genuine formulas, no injectivity of $\Rightarrow$, and no transitivity of $\preceq$.

This is particularly well suited to the project's permissive numerical encodings. The numerical instance can contain aliases or codes that do not decode as ordinary first-order syntax. The proof uses only the explicitly listed operations and closure conditions. Restricting the theorem to well-formed syntax would introduce an unnecessary invariant at this layer.

3.2 The unrestricted calculus is an instance

For the unrestricted syntactic calculus, define

$$x \preceq y \quad :\Longleftrightarrow \quad \mathsf{True}.$$

All three domination laws then have immediate proofs. Every unrestricted row is a restricted row for this instance, so the same core supplies K, S, Peirce, and composition. The `Lukasiewicz` module becomes a small adapter, while its public `Implicational` structure and its Tarski bridge keep their statements.

The existing six-goal derivation has 58 condensed-detachment inference steps, in addition to the restricted input row. Its renderer inlines two final aliases that no other derivation lemma uses, leaving 56 private derivation lemmas in the generated core. The important saving comes from sharing one derivation between the two consumers. This does not claim that 58 is optimal, or that a short generated line is necessarily cheap for the elaborator.

3.3 What this packet adds to the existing proposal

The source proposal comes from the repository's own commit `49e99c5b7`. This packet pins it, produces an exact patch against the current merged source, and checks its current consumers in an isolated component. The producer and neutral derivation are included, with a companion patch updating the current renderer, derivation input, and regeneration instructions. Regeneration of `DominatedImplication.lean` is byte-for-byte equal to the proposed file. The unrestricted adapter is a small maintained source module; it should not be overwritten by the older `-luk` command that renders a separate derivation.

The current numerical adapters `DomRho8` and `KCodeDom`, as well as the Tarski and `KThmK` consumers, are left intact in this component check. Named axiom reports are emitted for all six core exports and the four unrestricted basis exports. The validation ledger records the actual compiler result. A correct

untrusted producer still does not become part of the logical trust base: ordinary kernel-checked proof terms justify every generated application of modus ponens. An independent static review also found 14 existing public declaration headers unchanged, and byte-identical language definitions, unrestricted calculus interface, and complete Tarski namespace tail.

3.4 The Rocq choice

Two earlier Rocq renderings already offer a direct adoption path. The retained `CD222.v` has 230 lines and 22,865 bytes; its existing proposed condensed-detachment replacement has 129 lines and 12,007 bytes. The retained `DomRho8.v` has 1,169 lines and 22,673 bytes; its existing 52-step numerical replacement has 241 lines and 11,190 bytes. Together these selected sources fall from 45,538 to 23,197 bytes (49.1%). These are measured source sizes for existing alternatives, not a newly discovered derivation or a fresh whole-Rocq-endpoint build in this packet [10].

A representation-independent Rocq core is attractive for shared maintenance, but the smallest current numerical proof can be shorter than a generic core plus its adapter. Choose the objective explicitly. For the next modest production patch, the existing small numerical replacements have a simple integration story. For coordinated future changes across encodings and assistants, the shared neutral derivation and generic interface give a more uniform design. In either case, production adoption must update compile order, regeneration accounting, and the maintained Rocq endpoint gate.

4 A stronger invariant removes the dense bridge's second induction

4.1 The current duplication

`KDenseBridge` compares two representations of the same execution. The dense `Machine` uses a total step function whose halted state is absorbing. The sparse zipper machine uses a partial step function, where a missing transition is terminal. The local lemmas `rel_step` and `rel_halt` already prove the relevant transition and stopping correspondence [6].

The original `blank_iff` uses one induction to prove forward preservation, then a second induction to recover the dense run from a successful sparse prefix. The second proof is needed only because the first invariant's stopped alternative forgets that the dense machine has halted.

4.2 Retain the fact that was already available

Write D_t for the dense configuration after t total steps, z_0 for the sparse blank configuration, and $R(D_t, z)$ for the existing tape/state relation. Let H_D and H_Z be blank halting of the dense and sparse machines. Prove one invariant:

$$I(t): \quad (H_D \wedge H_Z) \ \vee \ \exists z \, \big(\mathsf{Run}_Z(t, z_0) = \mathsf{some} z \ \wedge \ R(D_t, z)\big).$$

At time zero, the right alternative is the existing blank relation. At a successor time, preserve the left alternative if it already holds. Otherwise inspect the state of D_t. If it is halted, time t itself witnesses H_D, and `rel_halt` together with the sparse prefix witnesses H_Z. If it is active, `rel_step` extends the exact synchronized prefix by one step.

Proposition 4.1 (One invariant gives both halting implications)**.** *The invariant $I(t)$ implies $H_D \leftrightarrow H_Z$ under the existing local transition and stopping lemmas.*

Proof. Suppose the dense machine halts at time t. Apply $I(t)$. Its stopped alternative already contains H_Z; its synchronized alternative ends at a configuration related to the halted dense state, so `rel_halt` supplies a sparse halt.

Conversely, suppose the sparse run reaches a terminal y after k steps. Apply $I(k+1)$. The stopped alternative supplies H_D. The synchronized alternative would make the sparse run successful for $k+1$ steps. But its first k steps end at y, and the next partial step is absent. This is a contradiction. □

This change preserves all existing definitions, imports, public theorem types, and the local tape-representation proof. The whole module drops from 193 to 170 lines and from 8,127 to 7,323 bytes. The `blank_iff` declaration itself drops from 52 to 29 lines and from 2,230 to 1,426 bytes. Audit commands live in a separate client, so these production-source measurements do not count added reporting commands.

The original and replacement modules were compiled separately against the same unchanged `Core` and `KZip` sources. An identical typed audit client checks the complete public theorem type in both cases. Both report the same axiom profile, `propext` and `Quot.sound`. This is a concrete source reduction that can be reviewed independently of all larger runtime and semantic proposals.

5 A shared simulation theorem with finite cleanup

The previous section is the best immediate local patch. A broader consolidation is also useful, but should be judged by its complete cost. The existing positive-stuttering proposals motivate a small weakening: a terminal source configuration may correspond to a target that still performs a finite epilogue. This weakening was already sketched in an earlier uncompiled repository proposal [7]; this packet provides checked Lean and Coq versions and concrete attachment experiments.

5.1 Precise partial-transition semantics

For a partial deterministic step function $f : X \to \mathrm{Option}(X)$, define

$$\begin{aligned} \mathsf{Run}_f(0,x) &= \mathsf{some} x, \\ \mathsf{Run}_f(n+1,x) &= \begin{cases} \mathsf{none}, & f(x) = \mathsf{none}, \\ \mathsf{Run}_f(n,x'), & f(x) = \mathsf{some} x', \end{cases} \\ \mathsf{Halts}_f(x) \quad &:\Longleftrightarrow \quad \exists n, d\, \big(\mathsf{Run}_f(n,x) = \mathsf{some} d \ \wedge\ f(d) = \mathsf{none}\big). \end{aligned}$$

Successful zero-length runs are allowed. Positive progress will therefore be an explicit condition on simulated source transitions.

Theorem 5.1 (Positive simulation with finite terminal cleanup)**.** *Let* $f : A \to \mathrm{Option}(A)$ *and* $g : B \to \mathrm{Option}(B)$*, and let* $R \subseteq A \times B$*. Assume:*

(i) Advance. *If* $R(a, b)$ *and* $f(a) = \mathsf{some}a'$*, there are* $k > 0$ *and* b' *such that* $\mathsf{Run}_g(k, b) = \mathsf{some}b'$ *and* $R(a', b')$*.*

(ii) Finish. *If* $R(a, b)$ *and* $f(a) = \mathsf{none}$*, then* $\mathsf{Halts}_g(b)$*.*

Then $R(a, b)$ *implies* $\mathsf{Halts}_f(a) \leftrightarrow \mathsf{Halts}_g(b)$*. The reverse implication uses only* Advance*.*

Lemma 5.2 (Cancellation at a terminal suffix)**.** *If* $\mathsf{Run}_g(n, b) = \mathsf{some}d$*,* $g(d) = \mathsf{none}$*, and* $\mathsf{Run}_g(k, b) = \mathsf{some}b'$*, then there is* m *with* $n = k + m$ *and* $\mathsf{Run}_g(m, b') = \mathsf{some}d$*.*

Proof. Induct on k. For $k = 0$, the successful prefix identifies $b' = b$. For $k + 1$, a terminal run of length zero would make the first step absent, contradicting success of the nonempty prefix. Thus both executions have a first step. Because g is a function, the first successors agree. Apply the induction hypothesis to their tails. □

Proof of the theorem. For preservation, induct on the length of a terminating source run. At a source terminal use *Finish*. At a source step use *Advance*, apply the induction hypothesis to the related successor, and prepend the finite target block.

For reflection, induct strongly on the target's number n of successful steps before its terminal configuration. If $f(a) = \mathsf{none}$, the source is already terminal. Otherwise *Advance* gives a successful target block of length $k > 0$. The terminal-suffix lemma supplies a remaining terminating run of length $m = n - k < n$. Apply the induction hypothesis to the related successors, then prepend the source step. The *Finish* hypothesis is absent from this argument. □

5.2 Why the hypotheses have exactly this shape

Finite cleanup is a real case. Take a source with one terminal state and a target that counts $2 \to 1 \to 0$ before stopping. Relate the source state to target state 2. *Finish* holds, although the target state related to the source terminal is not itself terminal. The Coq artifact includes this example as a proved theorem, together with failure of immediate terminal agreement.

Positive progress is necessary. Take a source with one state that loops and a target with one state that is already terminal. Every source step can be matched by a zero-length target run. The target halts and the source does not. Allowing $k = 0$ without a separate well-founded decrease would make reflection false. A bounded simulation trace cannot repair this logical gap.

Total absorbing transitions need an adapter. If a halted target is allowed to take arbitrarily many absorbing self-steps, those steps can incorrectly satisfy positive progress. The theorem is stated for partial steps. A total dense machine can be handled by a partialized configuration system or a time counter that stops at the first accepted time. The checked Lean experiment uses such an adapter. The smaller dense-bridge proof of Section 4 avoids introducing this extra infrastructure for that one client.

Source failure and source acceptance must be distinguished. For register interpreters, absence of a step can also denote a malformed or stuck configuration. K215's coverage and stopping lemmas

identify the valid terminal case at the relevant relation. The adapter must use those lemmas; it cannot simply replace the source's accepted-halting predicate by arbitrary stuckness.

5.3 Honest source accounting

The generic Lean theorem, zipper adapter, and register adapter allow real `KDenseBridge` and `KSim` clients to compile with a shared transport proof. However, in the measured two-client experiment the new utilities cost 219 lines and 8,543 bytes, while the clients together save 47 lines and 1,827 bytes. The combined source therefore grows by 172 lines and 6,716 bytes. It is useful checked infrastructure, not the immediate line-reduction patch recommended in Section 4.

Rocq already obtains a similar benefit through `BusyCoq.progress_halts` in `Sim230Main.v`. Its existing main simulation proof is about 23 proof lines. There is little reason to replace that successful factoring solely to make both assistants' file structures identical. The new standalone theorem is useful as a small common specification and for future transports; its Coq proof has no additional axioms.

6 Share exact decoder prefixes and certify every residual

6.1 The existing two halves

The current runtime proof already has substantial reusable machinery. The decoder certificate establishes exact tape behavior from a source path to a primitive-entry port. The existing site checker establishes the register operation and the return to dispatch. An earlier `proof-contracts` packet added semantic edge contracts and a generic prefix/suffix composition theorem, but compiled only the literal path `000`, covering two decrement outcomes. Its global result retained a universal `CheckedLeafSuffixes` premise [8].

Another prior packet independently replayed all residuals numerically, reporting 705,880 residual symbolic iterations against 926,351 whole-site iterations, a difference of 220,471 (23.8%) [9]. Those counts are useful motivation. They are not a measured Lean build speedup, and a Python replay does not discharge a Lean theorem's universal premise.

6.2 The missing attachment is a computable exact port

The semantic decoder theorem identifies a port through a relation `RoutedEntry`. A Boolean checker cannot directly choose an unspecified witness from that proposition. The new attachment follows the certified role graph along the entire prefix before the final leaf bit:

$$\texttt{followRoles}(\mathit{roles}, r, []) = \mathsf{some} r.$$

For $b :: P$, inspect role r and its b edge. An internal edge selects its recorded next role and continues along P; a missing role or premature primitive edge returns failure. The port state is read from the exact final role.

An induction on `RolePath` proves that this computable route equals every certified semantic route. If the same complete word is split as $P +\!\!+ [b] = P' +\!\!+ [b]$, list cancellation identifies $P = P'$. The computed port state consequently equals the port in `RoutedEntry`. This gives the equation

$$\begin{aligned}&\texttt{checkLeafSuffix}(M, P +\!\!+ [b], e)\\ &\quad = \texttt{checkEdgeFrom}(M, P +\!\!+ [b], \texttt{leafPort}(P, b, q), e)\end{aligned}$$

for every routed leaf. The new theorem `checkedLeafSuffixes_of_allSites` then turns one finite all-sites check into the earlier universal residual premise. This is an attachment of existing certified pieces, with a new computable selector and its correctness proof.

6.3 What remains occurrence-specific

The word passed to the suffix is the complete consumed source word. Its preceding bits remain on the left side of the zipper. Those bits affect carry, strip, break, continue, and overflow behavior. Thus equal physical entry states or equal short recent histories do not justify merging the associated proof obligations.

The residual check retains:

- both zero and nonzero decrement guards;
- the original logical-to-physical register permutation;
- path bounds, register bounds, gap growth, and overflow displacement;
- the exact successor path and register-file transformation;
- the spare-register and append-one behavior already covered by the box contracts;
- terminal state 230 and its separate no-transition hypothesis;
- strictly positive physical progress for every nonterminal source step.

The executor begins the residual proof with its progress flag set to false. It must establish positive suffix progress itself. This choice is slightly stronger than relying on a nonempty shared prefix, and makes the reflection condition independent of whether a particular prefix is empty. The interpretation quantifies over all natural gap lengths and valid register lists through the existing symbolic-soundness theorem; it is not a collection of bounded concrete tape tests.

6.4 Semantic coverage is the right interface

The existing `KSim.Covered` contains an equation saying that the old whole-site, fuel-limited Boolean checker returned true. A semantic prefix theorem plus a successful residual check establishes the required behavior. It does not imply that this other Boolean computation returned true.

The earlier semantic-interface proposal therefore remains an explicit prerequisite. The old Boolean coverage theorem becomes one producer of semantic edge contracts; the certified-prefix/residual route becomes another. Both feed the same register-to-TM simulation. A production endpoint can gain a semantic overload while retaining its old wrapper, or directly compose the new source-halting theorem with the unchanged dense bridge, boot theorem, and source-calculus calibration. No unproved conversion from semantic coverage back to an old checker equation is needed.

6.5 The finite check and its scope

The generated residual modules reuse the maintained partition into 105 source subtrees under the nine original batch names. Six larger batches import two halves each to bound compiler memory; none of the 105 equations changes. All 4,751 leaves and 6,176 guarded outcomes have now passed kernel checking. The source-tree binding, register-engine contracts, and literal terminal rows are also discharged.

The checked facade `KResidualSites215.source_halts_iff` proves

$$\texttt{RegisterPath.Halts p215}\ c \quad \longleftrightarrow \quad \texttt{KZip.Halts tm215}\ T$$

for every pair satisfying `KSim.Rel p215` $c\ T$. There is no remaining box-contract or universal residual premise at this interface. The final `covered` and `source_halts_iff` audits use only `propext`, `Classical.choice`, and `Quot.sound`.

The final component contains 51 project modules, including 24 new modules, and completed across source-bound resumed compiler invocations. The reproducible driver uses one Lean worker, a 3 GiB process limit, and explicit recursion and heartbeat limits. Accepted logs and failed development attempts are distinguished in the evidence. This is narrower than a fresh rebuild of the entire machine-to-ZFC endpoint. It also increases source and import count at first: the semantic `KSim` interface and routing adapter add infrastructure, and old full-site proofs may remain during migration.

7 Rocq: extract the right common layer

7.1 Dense relabeling across the family

Eight retained dense-machine files, for state counts 215, 217, 218, 221, 225, 226, 227, and 229, contain the same 74-line, 2,883-byte proof block. It starts at `dense_step_sparse` and ends at `dense_sparse_halts_iff`. The copies are emitted from a common `Dense229.v` template. Accordingly this is generated-source and elaboration duplication, not eight separately hand-maintained mathematical arguments.

A generic relabeling functor can parameterize the dense and sparse state types, the expansion map, transition commutation, and initial-state compatibility. It proves the tape-preserving simulation and halting equivalence once. A concrete `Dense215` file then supplies its literal table and finite row checks. The new component experiment and its validation status are included in the archive. The concrete adapter is 92 lines and 4,226 bytes, compared with 155 lines and 6,725 bytes originally. The shared module adds 111 lines and 3,966 bytes, so this single-client experiment grows by 48 lines and 1,467 bytes overall. Both the standalone cleanup theorem and the concrete relabeling component pass compilation and recursive `coqchk` with Coq 8.18.0. The concrete halting theorem is closed under the global context. The imported library has its existing global assumptions; the theorem-specific assumption report shows they are not used by this endpoint.

For a single selected endpoint, the generic source plus adapter may be longer than one in-place copy. Its main benefit is a smaller common specification and reduced family-wide generated output. Keep these claims separate from the local net saving of the Lean one-invariant patch.

7.2 Prioritize adoption rather than another abstraction

The most direct Rocq reductions are already available as proposed artifacts: the smaller condensed-detachment derivations, normalized source correspondence, and one-sided finite checking with a mathematical symmetry theorem. Lean has already adopted several corresponding interface improvements. A controlled Rocq adoption should select one change at a time, update the generator, compile the affected clients, and run the maintained recursive kernel check.

The suggested order is:

1. integrate the smaller implication source files and their producer wiring;

2. port the mature source-certificate symmetry/normalization interface;
3. use a generic relabeling module if multiple maintained variants will consume it;
4. share decoder primitive/return contracts only after the exact continuation state and progress obligations have been represented.

There is no need to rewrite Rocq's already short `progress_halts`-based simulation proof solely to resemble Lean's implementation.

8 Validation, trust, and reproducibility

8.1 Validation categories

The archive distinguishes mathematical proof, compiler checking, finite data checks, and source accounting. The generic cleanup theorem is proved in both assistants. The immediate Lean bridge patch has a matched baseline/candidate theorem-signature and axiom audit. The shared implication proposal has a source-pinned producer and current-caller component build. Decoder residual checking has a separate ledger.

No new `sorry`, `admit`, `Admitted`, declared proof axiom, unchecked native-decision shortcut, or disabled termination/positivity check is used in the delivered proof changes. Hashes identify inputs and outputs; they do not establish the correctness of a mathematical theorem. The proof-assistant commands and their actual logs are the evidence for kernel checking.

Lean checks use version 4.32.0, the repository's pinned version. The standalone Coq checks use Coq 8.18.0, whereas production Rocq adoption still has the maintained 9.1.0 gate. Compatibility with the older Coq compiler is useful component evidence; it must not be relabeled as a production Rocq 9.1 endpoint build.

Build timings, when present in receipts, are descriptive local observations. They exclude queue waiting where the driver records compiler time separately. Concurrent work and different caches can affect them. This report claims neither a paired whole-endpoint speedup nor an optimal number of proof steps.

8.2 A disciplined production merge

For the immediate patch, apply the two independent source changes to the pinned revision, retain the generator input for the shared implication core, and rebuild the affected users. Print the public theorem types and axiom profiles. Then run the exact `KEndpoint215` client with dependency invalidation based on the changed sources. The existing endpoint theorem statement should remain byte-for-byte or elaboration-equivalent at the public interface.

For the decoder proposal, first preserve the old executable checker definitions and provide semantic adapters. Compile the exact port selector, source binding, and every residual batch. Attach semantic coverage to the endpoint through an explicit overload. Only after that integration passes should redundant old whole-site proofs leave the selected closure. Retaining the old route during review provides a useful compatibility control, but retaining both routes permanently may obscure the intended simplification.

For Rocq, make generator and compile-order changes concrete in the same patch as the proof-source changes. Rebuild the current endpoint and recursively check it with the production toolchain. Earlier receipts from another source revision can guide review; they cannot substitute for these final gates.

9 Further reductions worth pursuing

Write each subtree path once. The 105 maintained finite-check statements print both a path P and its reversed prefix. Introduce the definitional abbreviation

$$\mathsf{CheckedSubtree}(\mathit{test}, t, P) := \big(\texttt{allSites}\ \mathit{test}\ \mathrm{reverse}(P)\ (\texttt{descend}\ t\ P) = \mathsf{true}\big).$$

Each statement then carries one path literal; the checker receives precisely the same arguments after unfolding. An exact static rewrite of the nine maintained batch files reduces their combined size from 55,794 to 36,991 bytes: 18,803 bytes of repeated statement text. The abbreviation and its comment add 238 bytes to the existing shared module, leaving a net saving of 18,565 bytes across the ten selected files. The separate proposal includes a reproducer and companion generator patch, and labels the Lean changes as uncompiled. The ten-file metric excludes the generator change. It changes neither the 105 finite obligations nor their evaluator cost. This is a direct readability and generated-source improvement that can be reviewed independently of the residual architecture.

Factor return paths only after specifying their complete context. The shared Home return proof is already a useful precedent. Further primitive and return factoring should use contracts on a parameterized full continuation, not a coarse entry-state classification. A useful contract describes the entire list prefix, gap equation, source path operation, register update, and progress witness.

Separate proof generation from proof representation. The condensed-detachment search may use aggressive heuristics, deletion, and unification. Its output should remain a neutral derivation with exact typed replay. Keep one generator input for logically shared facts. Avoid compressing a file only by moving a large opaque computation into a checker if that makes error localization or kernel cost worse.

Use one invariant before introducing a new framework. The dense-bridge patch illustrates a particularly inexpensive kind of simplification: strengthen an already proved invariant with information present at the transition where it is established. The stronger proposition can eliminate a later induction. Other bidirectional compiler and interpreter proofs are natural places to search for the same pattern: the forward proof may already know the witness needed by the reverse direction and simply discard it.

Measure the theorem's selected closure, not all historical variants. The repository intentionally contains old machines, alternative proofs, and research packets. Removing or sharing those files can simplify maintenance without changing the K215 proof's dependency closure. Conversely, moving a general semantic theorem behind a narrow import can substantially reduce the closure while leaving the repository's total source unchanged. Report both quantities with their scope.

Keep the endpoint and the internal formalization projects separate. The next simplification patch should preserve the existing external machine/refutation equivalence and its literal table binding. A future internal PA or ZFC formalization will need its own coding and proof-translation obligations. Treating that separate task explicitly will make both projects easier to verify.

A Artifact guide

Path	Contents and purpose
`report.tex`, `report.pdf`	This note, with its two included TEX files, a `Makefile`, and pinned primary-source references.
`shared-basis/`	Two proof-source replacements, a separate producer/input and documentation patch, 16 isolated Lean sources, the neutral derivation, renderer, reproduction script, static interface review, and successful component receipt and logs.
`lean-bridge/`	The recommended small dense-bridge patch and a distinct optional finite-cleanup consolidation. Includes actual modules, typed audit, boundary examples, fresh-build driver, source measurements, and logs.
`decoder-residual/`	Exact role-path selector and attachment proof, occurrence-specific finite certificates, semantic runtime facade, source pins, partition provenance, build driver, and a separate validation ledger.
`rocq-components/`	Constructive finite-cleanup proof, generic relabeling experiment, concrete Dense215 adapter, reproduction instructions and receipts, plus the adoption map and exact patch for the existing implication proposals.
`subtree-abbreviation/`	Independent uncompiled proposal and reproducible source estimate for eliminating duplicated path literals from the 105 maintained finite-check statements.
`evidence/`	Current endpoint source closure, measured source comparisons, and additional review records. These are scoped measurements, not substitute proof certificates.
`SHA256SUMS.json`	Whole-packet file integrity manifest. Recheck it with `verify_integrity.py`; it excludes itself and compiled reproduction outputs.
`NOTICE.md`, `LICENSE`	Attribution, origin commits, scope, and the project's MIT No Attribution license.

Start with the independent production-source patches. At the root of the pinned checkout, run `git apply -check` on `shared-basis/shared-basis.patch`, `shared-basis/producer-wiring.patch`, and `lean-bridge/minimal.patch` using their actual extracted paths. These patches were checked together without changing the production checkout. The optional `lean-bridge/terminal-drain.patch` is an alternative to the minimal bridge patch and must not be stacked on it. The cleanup experiment and the residual architecture also use different `KSim` variants; their independent component builds do not certify a combination of those variants.

Reproduce the checked components. Each component's README gives the precise command and required compiler. The shared basis includes its entire 16-module source component; the bridge and runtime drivers read hash-pinned prerequisites from a local checkout. No compiler binaries or precompiled project objects are in the archive. Lean checks use version 4.32.0. Coq component checks use 8.18.0; the production Rocq 9.1.0 gate remains a separate adoption step.

Rebuild the report. Run `make report` from this directory with a TeX installation containing the packages listed at the start of `report.tex`. This runs pdfLaTeX twice so that references and page destinations settle. The report was also rendered to page images and visually inspected before release.

B Primary source references

All repository references below are to the reviewed project or its explicitly identified prior proposal. Paths in the source inventory are authoritative for the pinned revision. The new code and validation receipts in this archive are primary evidence for this packet's own component results.

References

[1] ProveIt, reviewed source tree. https://github.com/VladimirReshetnikov/ProveIt/tree/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC

[2] `KEndpoint215.lean`, exact machine-specific endpoint. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean

[3] `KEndpoint.lean`, generic endpoint composition. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint.lean

[4] Twelfth K215 proof-simplification checkpoint. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase12/README.md

[5] Existing shared-basis renderer and source proposal, commit `49e99c5b7`. https://github.com/VladimirReshetnikov/ProveIt/commit/49e99c5b7cc6c2e164331961de28bad3904361fd

[6] `KDenseBridge.lean`, current dense/sparse proof. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KDenseBridge.lean

[7] Earlier finite-cleanup contract proposal. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/further-simplifications/terminal_contract/README.md

[8] Earlier semantic proof-contract and prefix/suffix component packet. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/proof-contracts/README.md

[9] Earlier shared-interface and finite residual audit packet. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/sharing-interfaces/README.md

[10] Existing shorter Rocq implication sources and prior validation scope. https://github.com/VladimirReshetnikov/ProveIt/blob/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5/Computability/BusyBeaver/Research/ZFC/k225-formal/rho8cd/rocq-proposal/README.md